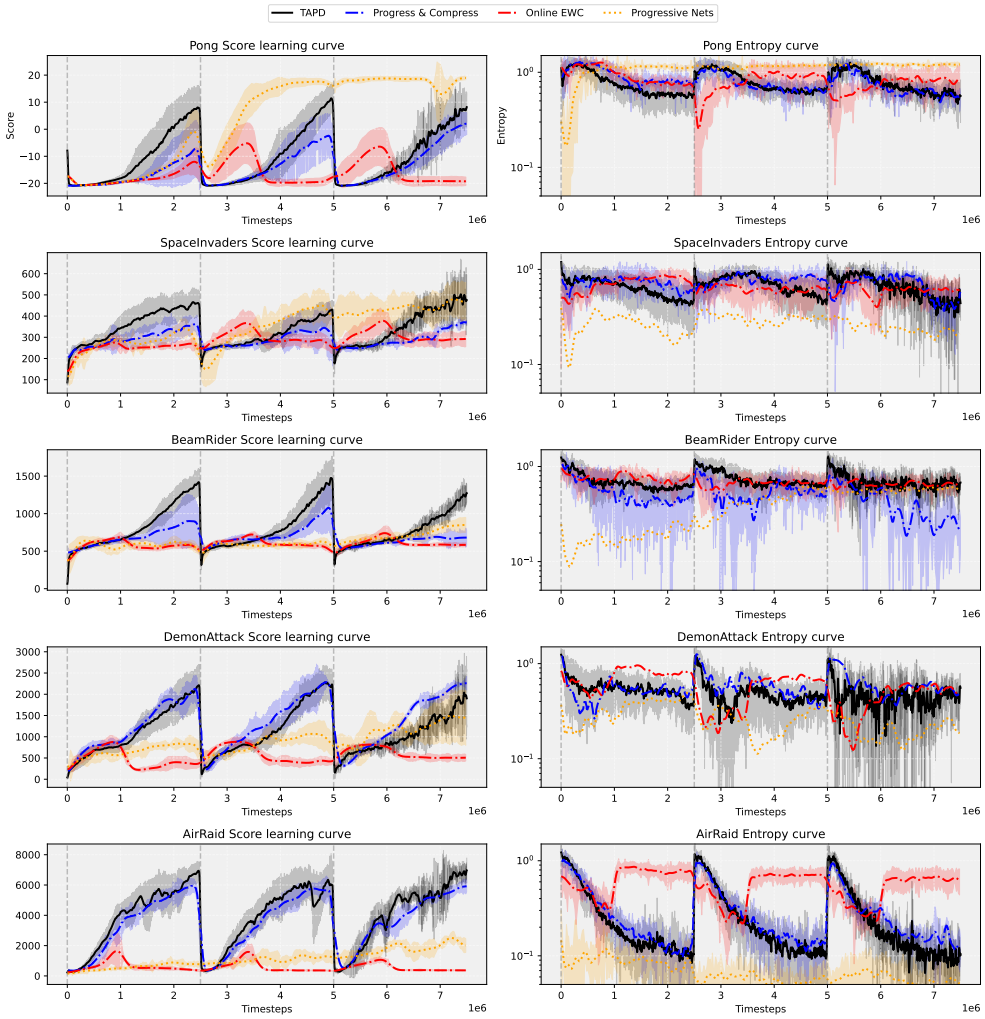

Fig. 4: The learning curves depicted represent the obtained rewards in the progress phase, against Task Agnostic Policy Distillation (TAPD), Online EWC, Progressive Nets, and the reproduced Progress & Compress baseline. Reading from left to right, both performance and entropy are plotted. Tasks are learned in a sequential manner in the following order: Pong, SpaceInvaders, BeamRider, DemonAttack, and AirRaid. TAPD utilizes the distilled knowledge from the task-agnostic phase. Results are averaged over 4 seeds and reflect the averages of scores taken across 100 episodes. Each task is revisited three times (gray vertical lines), allowing for training for 2.5M timesteps on each visit in the progress phase.

parameters, indicating a robust growth mechanism. However, despite the lack of interference in Progressive Nets, the TAPD algorithm ultimately converges to a higher performance by the end of the task visits. This suggests a positive forward transfer and a recurring pattern of stable entropy, contributing to its success.

In the BeamRider task, the TAPD algorithm consistently outperforms all other algorithms throughout the visits. Despite the Progressive Nets algorithm's use of dedicated task parameters, its performance and sample efficiency remain low, suggesting that lateral connections between columns and dedicated task parameters do not necessarily enhance forward transfer. In the AirRaid task, the Progress & Compress baseline shows comparable performance to TAPD during the initial visits, but TAPD ultimately surpasses it by the end of the training. This suggests that the distillation of the intrinsically motivated behavior during the task-agnostic phase serves as a strong regularizer, enabling TAPD to achieve higher final performance. The smaller gap between visits in catching up to earlier performance suggests greater sample efficiency. In contrast, Progressive Nets algorithm struggles to catch up with the final performance of TAPD, further highlighting the advantages of the TAPD approach. In terms of entropy, it is evident that the TAPD algorithm begins with exploratory behavior that gradually decreases over the course of visits. This pattern suggests that the agent initially explores the environment more extensively compared to the more conservative action selection process of the Progressive Nets, which may contribute to its slower rate of performance improvement. Injecting task-agnostic behavior into the knowledge base during the task-agnostic phase appears to lead to a better trade-off between exploration and exploitation, enhancing overall performance and demonstrating that task-agnostic policy distillation indeed facilitates positive forward transfer.

Moreover, TAPD is significantly more scalable than Progressive Nets. TAPD requires only two networks, while Progressive Nets face a major limitation: as tasks are added, the network size increases substantially. Specifically, the number of hidden units and feature maps in Progressive Nets grows linearly with the number of columns, and the number of parameters grows quadratically. This scalability issue makes TAPD a more efficient and practical solution, especially when dealing with a large number of tasks. Additionally, Progressive Nets require the specific task ID during training to query the correct column in subsequent visits. This dependency on task identification is not necessary in TAPD, which makes it a more versatile method. TAPD can quickly adapt to tasks during the initial phases of training without needing explicit task IDs, further underscoring its flexibility and robustness compared to Progressive Nets.

5.4 Assessing Forward Transfer

Positive forward transfer refers to improved performance on a new task immediately following the learning of previous tasks, indicating an algorithm's ability to leverage prior knowledge effectively. Two key indicators demonstrate positive forward transfer: (1) the performance of the active column across tasks over multiple visits, which captures improvement in learning a new task after exposure to prior tasks as well as gains from repeated visits to an old task, and (2) the average normalized performance of each algorithm across tasks, with high values suggesting successful application of previously acquired knowledge. Table 2 presents the performance data of the active column network across different tasks over three visits, comparing TAPD, Online EWC, Progressive Nets, and Progress & Compress. This comparison evaluates how learning on one task influences subsequent task performance. In the table, an upward arrow (↑) indicates improved performance compared to the previous visit, while a

downward arrow (↓) indicates decreased performance. Additionally, Fig. 5 illustrates the performance trends across multiple tasks and visits. Performance was evaluated using normalized scores, with further analysis focusing on the variance in performance across tasks and visits. As shown in Table 2, TAPD outperforms all other algorithms in the three visits for the majority of tasks. In the third visit, it maintains the highest scores for Task 4 and Task 5, despite a slight decline in performance compared to the previous visit, as was also observed for Task 5 in the second visit.

Table 2: Performance of the active column network on subsequent tasks after visiting previous tasks. Tasks 1-5 were visited in the following order: Pong, SpaceInvaders, BeamRider, DemonAttack, and AirRaid, respectively. Results are averaged over 8 seeds.

	Task 1	Task 2	Task 3	Task 4	Task 5
First Visit					
Progress & Compress (Active-Col)	-4.25	327.41	732.12	1029.89	2468.09
TAPD (Active-Col)	14.86	**439.62**	**1441.82**	**2479.65**	**2605.39**
Online EWC	12.14	248.85	528.377	209.62	505.30
Progressive Nets	**16.94**	437.3	646.17	910.16	1279.05
Second Visit					
Progress & Compress (Active-Col)	14.56 ↑	335.97 ↑	877.66 ↑	963.57 ↓	2529.31 ↑
TAPD (Active-Col)	**18.51** ↑	**472.91** ↑	**1537.52** ↑	**2699.95** ↑	**2580.15** ↓
Online EWC	-14.71 ↓	275.79 ↑	606.64 ↑	460.08 ↑	433.57 ↓
Progressive Nets	-3.4 ↓	618.7 ↑	697.6 ↑	612.37 ↓	1455.93 ↑
Third Visit					
Progress & Compress (Active-Col)	15.04 ↑	387.19 ↑	892.40 ↑	1034.01 ↑	2501.98 ↓
TAPD (Active-Col)	19.94 ↑	**483.87** ↑	**1550.30** ↑	**2383.18** ↓	**2506.41** ↓
Online EWC	-17.19 ↓	326.31 ↑	641.08 ↑	592.97 ↑	390.29 ↓
Progressive Nets	**20.33** ↑	457.5 ↓	864.68 ↑	1084.24 ↑	2312.06 ↑

The variance across visits (top plot in Fig. 5) illustrates the variance of each algorithm's performance across multiple visits, providing insight into the stability of their learning processes. A high variance indicates that the algorithm's performance fluctuates significantly from one visit to the next, suggesting instability or sensitivity to specific conditions during each visit. Conversely, low variance indicates a more stable learning process, with the algorithm performing consistently and similarly across different visits. Progress & Compress demonstrates the lowest variance, reflecting a stable performance across visits, making it potentially more reliable in scenarios requiring consistent outcomes. In contrast, the Progressive Nets algorithm shows the highest variance over visits, indicating significant performance fluctuations and suggesting overfitting during each task visit.

The variance across tasks (middle plot in Fig. 5) measures how consistently each algorithm performs across different tasks. Progress & Compress and Progressive Nets exhibit higher variance, indicating that their performance is uneven across tasks. This suggests that these algorithms may excel in certain types of tasks but struggle with others, leading to a less predictable overall performance. High variance is a sign of overfitting to specific task characteristics, limiting the algorithms generalization capabilities across diverse tasks. This outcome is expected, as the model's complexity